

APUNTES

Víctor Manuel Peiró Martínez
PROGRAMACION WEB 2 - SERVIDOR FIIS 3ºA

Contenido

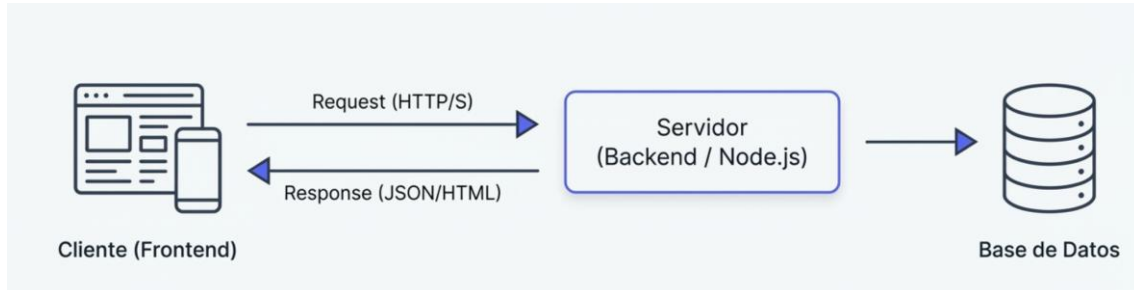
Tema 1 – Fundamentos Modernos del Desarrollo Backend.....	4
1 – Introducción al Backend Web.....	4
2 – Node	4
3 – Configuración.....	5
4 – Sistema de Módulos	5
5 – I/O Asíncrono con Promesas	6
6 – Módulos Core de Node.....	7
7 – Gestión de Paquetes.....	8
8 – Tooling Nativo y Experiencia de Desarrollo	8
Tema 2 – Eventos y Asincronía en Node.....	9
1 – Event Loop: Arquitectura Moderna.....	9
2 – Microtasks vs Macrotasks	9
3 – EventEmitter.....	9
4 – Promises y Async/Await.....	9
5 – Manejo de Errores	10
Tema 3 – HTTP y Enrutamiento.....	11
1 – Modelo Cliente-Servidor	11
2 – Protocolo HTTP	11
3 – Objeto Request y Response	12
4 – Anatomía de una URL.....	12
5 – CORS.....	13
Tema 4 – Framework Express	14
1 – Introducción Express.....	14
2 – Primera Aplicación Express 5	14
3 – Routing	14
4 – Routers Modulares	15
5 – Middleware.....	16
6 – Validación con Zod.....	16
7 – Servir Archivos Estáticos y Logging.....	17
Tema 5 – MVC y BBDD NoSQL con MongoDB.....	18
1 – Patron MVC para APIs.....	18
2 – MongoDB y Mongoose.....	18

3 – CRUD con MongoDB	19
4 – Relaciones Entre Documentos.....	19
5 – Subida de Archivos con Multer	20
Tema 6 – Validación Avanzada y Manejos de Errores	21
1 – Validación Avanzada con Zod	21
2 – Soft Delete (Borrado Lógico).....	22
3 – Sanitización de Datos	22
4 – Rate Limiting.....	22
5 – Seguridad Con Helmet	22
Tema 7 – Autenticación y Autorización con JWT	23
1 – Introducción a JWT	23
2 – Hash de Contraseñas	23
3 – Manejo de JWT	23
4 – Middleware de Autenticación.....	23
5 – Autorización de Roles	23
6 – Refresh Token.....	24
Tema 8 – Documentación, Testing y Monitorización.....	25
1 – Documentación con Swagger.....	25
2 – Testing Con Jest.....	26
3 – Monitorización Con Slack.....	26
Tema 9 – Supabase y PostgreSQL.....	27
1 – Introducción a Supabase.....	27
2 – PostgreSQL vs MongoDB	27
3 – Prisma: El ORM Moderno	27
4 – Modelo de Datos	28
5 – CRUD.....	28
6 – Relaciones en PostgreSQL	29
7 – Migraciones y Prisma Studio	29
Tema 10 – WebSockets – Comunicación IRT	30
1 - ¿Qué son los WebSockets?.....	30
2 – HTTP vs WebSocket.....	30
3 – Socket.IO.....	30
4 – Rooms y Namespaces.....	31

Tema 11 – Deploy y DevOps.....	32
1 – Preparar para Producción	32
2 – Docker	32
3 – Docker Compose	33
4 – CI/CD con GitHub Actions	34
Tema 12 – TypeScript con Node.js.....	35
1 – TypeScript.....	35
2 – Tipos Básico	35

Tema 1 – Fundamentos Modernos del Desarrollo Backend

1 – Introducción al Backend Web



Arquitectura Cliente-Servidor: El cliente hace peticiones que el servidor gestiona

- **Cliente:** Dispositivo que solicita los recursos
- **Servidor:** Máquina que procesa peticiones y envía respuestas
- **Protocolo:** Reglas de comunicación entre ambos

Frontend vs Backend vs Full Stack

Area	Responsabilidad	Tecnologías Típicas
Frontend	Interfaz de Usuario, Interacción	React, Vue, HTML/CSS/JS
Backend	Lógica de Negocios, APIs, datos	Node.js, Express, Bases de Datos
Full-Stack	Ambas	Todo lo anterior

Propósito del Servidor Backend:

- Procesa peticiones HTTP
- Autentica y autoriza peticiones
- Valida datos de entrada
- Ejecuta lógica de negocio
- Interactúa con bases de datos
- Responde con datos

2 – Node

Node: Entorno de ejecución de JavaScript. Basado en el motor V8 de Chrome.

Características:

- **Asíncrono y orientación de eventos:** No bloquea operación I/O
- **Single-Threaded con Event Loop:** Un hilo principal + pool workers
- **Motor V8:** Compila JS a código máquina
- **Ecosistema npm:** El mayor registro de paquetes del mundo

Event Loop: Coordina todo, permitiendo alta concurrencia sin bloqueos. Prioridad:

1. Código Sincrono
2. Microtasks (Promises, process.nextTick)
3. Macrotasks (setTimeout, setInterval, I/O)

3 – Configuración

Como inicializar un proyecto: npm init -y. -y para configuración por defecto

Al crear un proyecto se crea el fichero package.json:

```
{
  "name": "mi-api",
  "version": "1.0.0",
  "description": "API REST con Node.js",
  "type": "module",
  "main": "src/index.js",
  "scripts": {
    "dev": "node --watch --env-file=.env src/index.js",
    "start": "node --env-file=.env src/index.js",
    "test": "node --test"
  },
  "engines": {
    "node": ">=22.0.0"
  },
  "keywords": [],
  "author": "Tu nombre",
  "license": "MIT"
}
```

- “type”: “module” : Activa ESM (ECMAScript Modules)
- “engines”: Especifica versión mínima de Node
- “--watch”: Reinicio Automático Nativo
- “--env-file”: Carga de variables de entorno nativas

4 – Sistema de Módulos

Característica	CommonJS (Legado)	ESM (Moderno)
Sintaxis	require() / module.exports	import / export
Carga	Síncrona, bloqueante	Asíncrona
Análisis	Dinámico (runtime)	Estático (permite tree-shaking)
Top-Level Await	No	Si (async await)
Compatibilidad	Solo Node.js	Node.js + Navegadores

Usamos ESM como estándar

Sintaxis:

- **Exportar:**

```
//Exportar normal
export const sumar = (a,b) => a+b
export const restar = (a,b) => {return a-b} //Return solo si llaves

//Exportar Default
export default class Calculadora {

  constructor(a,b) {
    this.num1 = a
    this.num2 = b
  }

  sumar() {
    return this.num1 + this.num2
  }

  restar() {
    return this.num1 - this.num2
  }
}
```

- **Importar:**

```
import { sumar, restar } from "./math.js"; // Sino default, entre llaves
import Calculadora from "./math.js";
```

Top-Level Await: Se puede usar await fuera de funciones async

5 – I/O Asíncrono con Promesas

Problema: Node usaba callbacks error-first

Solución: async await con promesas

```
import { readFile, writeFile } from 'node:fs/promises';

try {
  const data1 = await readFile('archivo1.txt', 'utf-8');
  const data2 = await readFile('archivo2.txt', 'utf-8');
  await writeFile('resultado.txt', data1 + data2);
  console.log('Completado');
} catch (error) {
  console.error('Error:', error.message);
}
```

Operaciones en Paralelo:

```
// Secuencial (lento): espera cada archivo
const data1 = await readFile('archivo1.txt', 'utf-8');
const data2 = await readFile('archivo2.txt', 'utf-8');

// Paralelo (rápido): lee todos a la vez
const [file1, file2, file3] = await Promise.all([
  readFile('archivo1.txt', 'utf-8'),
  readFile('archivo2.txt', 'utf-8'),
  readFile('archivo3.txt', 'utf-8')
]);
```

6 – Módulos Core de Node

Usar siempre node: antes del paquete

- Evita dependency confusión (paquetes con el mismo nombre maliciosos)
- Mas rápido (no busca en node_modules)
- Claridad (sabes que es un modulo core)

node:fs/promises : Sistema de Archivos

- **readFile(nombre,encoding)**: Lectura de Archivo. Devuelve un texto en formato JSON
- **writeFile(nombre, datos)**: Escritura de Datos
- **mkdir(rutaDirectorio,{recursive:true})**: Creación de directorios, es recursivo
- **readdir(directorio)**: Listado de archivos de un directorio
- **stat(nombre)**: Información Archivo
- **unlink()**: Eliminar Archivo
- **rename(viejo,nuevo)**: Renombrar archivos

node:path : Rutas de Archivos

- **path.join()**: Unión de Rutas
- **path.resolve()**: Rutas Absolutas
- **path.dirname()**: Nombre directorio
- **path.basename()**: Nombre archivo
- **path.extname()**: Extension del archivo

__dirname y **__filename**: Se puede usar import.meta.dirname/filename

node:os : Para sistema operativo

node:crypto: Criptografía

- **randomUUID()**: Generación de UUID v4
- **randomBytes(n)**: Bytes Aleatorio
- **createHash('sha-256')**: Hash SHA-256

node:process : Proceso Actual

- **process.env**: Variables de Entorno
- **process.argv**: Argumentos línea de comandos
- **process.cwd()**: Directorio de trabajo

Timers:

- **setTimeout(función, X)**: Ejecuta una función después de X ms
- **clearTimeout()**: cancela el timeout
- **setInterval()**: Ejecutar cada intervalo
- **clearInterval()**: Cancela el intervalo

7 – Gestión de Paquetes

Semver: Versionado Semántico – MAJOR.MINOR.PATCH

- **Major:** Cambios incompatibles
- **Minor:** Nuevas funcionalidades
- **Patch:** Corrección de Bugs

Rangos de Versión:

- **^:** Compatible con MINOR
- **~:** Compatible con PATCH
- **>=:** Mayor o igual
- **Sin Símbolo:** Version Exacta

node_modules: Carpeta con las dependencias instaladas

8 – Tooling Nativo y Experiencia de Desarrollo

--watch: Reinicia automáticamente al detectar cambios

Archivo .env:

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://localhost:5432/mydb
JWT_SECRET=mi-secreto-super-seguro
```

Tema 2 – Eventos y Asincronía en Node

1 – Event Loop: Arquitectura Moderna

Las 6 fases:

1. **Timers:** Callbacks de `setTimeout` y `setInterval`
2. **Pending Callbacks:** Callbacks de I/O diferidos de la iteración anterior
3. **Idle, prepare:** Solo uso interno
4. **Poll:** Nuevos eventos de I/O, ejecuta callbacks de I/O
 - Calcula cuanto tiempo bloquear esperando I/O
 - Procesa eventos de la cola poll
 - Si no hay timers programados, espera indefinidamente por I/O
 - Si hay timers vuelven a la fase timers cuando expiren
5. **Check:** Callbacks de `setImmediate()`
6. **closeCallbacks:** Callbacks de cierre

2 – Microtasks vs Macrotasks

Jerarquia:

1. **Código Síncrono:** Call Stack
2. **Maxima Prioridad Asíncrona:** `process.nextTick()`
3. **Microtasks:** `Promise.then()`, `queueMicrotask()`
4. **Macrotasks:** `setTimeout()`, `setImmediate()`, I/O

3 – EventEmitter

Metodos:

- **.on(evento, callback):** Registrar listener
- **.once(evento, callback):** Listener que se ejecuta una vez
- **.emit(evento, ...args):** Emite un evento con argumentos
- **.off(evento, callback):** Elimina un listener
- **.removeAllListeners(evento):** Elimina todos los listeners
- **.listenerCount(evento):** Lista todos los listeners

4 – Promises y Async/Await

Sintaxis de Promesas: `new Promise((resolve,reject) => {})`

Estados de Una Promesa:

- **resolve(valor):** Promesa Cumplida
- **reject(error):** Promesa Rechaza

Se consume la promesa con `await` y usar `try/catch`

Metodos:

- **Promise.all([...]):** Rechaza si cualquiera falla. Devuelve array o primer error
- **Promise.allSettled([...]):** Espera todas, rechaza nunca. Devuelve {estado, valor}
- **Promise.any([...]):** Primera exitosa gana. Devuelve valor o Aggregate Error
- **Promise.race([...]):** Primera que se resuelve o rechaza. Devuelve valor o error

5 – Manejo de Errores

Error.cause: Encadenar errores

```
async function fetchUsuario(id) {
  try {
    const response = await fetch(`/api/usuarios/${id}`);
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    return response.json();
  } catch (error) {
    // Encadenar error original como "cause"
    throw new Error(`No se pudo cargar usuario ${id}`, { cause: error });
  }
}

// Acceder a la cadena de errores
try {
  await fetchUsuario(123);
} catch (error) {
  console.log(error.message); // "No se pudo cargar usuario 123"
  console.log(error.cause?.message); // "HTTP 404"
}
```

AggregateError: Multiples Errores

```
// Promise.any lanza AggregateError si todas fallan
try {
  await Promise.any([
    fetch('https://api1.com/datos'),
    fetch('https://api2.com/datos'),
    fetch('https://api3.com/datos')
  ]);
} catch (error) {
  if (error instanceof AggregateError) {
    console.log('Todas las APIs fallaron:');
    error.errors.forEach((e, i) => {
      console.log(`  API ${i + 1}: ${e.message}`);
    });
  }
}
```

Clases de Errores Personalizadas:

```
class AppError extends Error {
  constructor(code, options = {}) {
    super(AppError.messages[code] ?? code, options);
    this.code = code;
    this.name = 'AppError';
    this.timestamp = new Date().toISOString();
  }

  static messages = {
    USER_NOT_FOUND: 'Usuario no encontrado',
    INVALID_INPUT: 'Datos de entrada inválidos',
    DB_CONNECTION_FAILED: 'Error de conexión a base de datos'
  };
}

// Uso
throw new AppError('USER_NOT_FOUND', { cause: originalError });
```

Tema 3 – HTTP y Enrutamiento

1 – Modelo Cliente-Servidor

Cliente: Aplicación que realiza peticiones al servidor

Servidor: Programa que se ejecuta en maquina para ofrecer servicios a la maquina

Protocolo: Conjunto de reglas que permiten transmitir información entre dispositivos

2 – Protocolo HTTP

Solicitudes: Mensaje que envía el cliente al servidor

Componentes de Una Solicitud:

- **Método:** Acción
 - **Método Idempotente:** Ejecutar el método múltiples veces tiene el mismo resultado que ejecutarlo una solo
- **URL/Path:** Dirección del recurso
- **Versión:** Versión de HTTP
- **Headers:** Metadatos Adicionales
- **Body:** Datos enviados al servidor

Métodos:

- **GET:** Obtener un recurso
 - No tiene body
 - Es idempotente
- **POST:** Crear un recurso nuevo
 - Tiene body
 - No es idempotente
- **PUT:** Reemplazar un recurso completo
 - Tiene body
 - Es idempotente
- **PATCH:** Modificar parcialmente un recurso
 - Tiene body
 - No es idempotente
- **DELETE:** Eliminar un recurso
 - Puede tener body
 - Es idempotente

Respuesta: Es el mensaje que devuelve el servidor

Componentes de Una Respuesta:

- **Código de Estado:** Numero que indica resultado
- **Texto de Estado:** Descripción de Código
- **Headers:** Metadatos de respuesta
- **Body:** Datos devueltos

Códigos de Estado:

- **1xx:** Informativos
- **2xx:** Éxito
 - **200:** OK
 - **201:** Created
- **3xx:** Redirección
- **4xx:** Error de Cliente
 - **400:** Bad Request
 - **401:** Unauthorised
 - **403:** Forbidden
 - **404:** Not Found
- **5xx:** Error de Servidor
 - **500:** Internal Error

3 – Objeto Request y Response

Request (req):

- **req.method:** Método de petición
- **req.url:** URL
- **req.headers:** Cabeceras

Response (res):

- **res.statusCode:** Código de respuesta
- **res.setHeader():** Establecer header
- **res.writeHead():** Establecer código de respuesta y estado
- **res.end():** Enviar estado y cerrar conexión

4 – Anatomía de una URL

Ejemplo: <https://www.ejemplo.com:8080/users/123?active=true&sort=name#section1>

- **https:** Protocolo
- **www.ejemplo.com:** Dominio/host
- **8080:** Puerto. Es opcional
- **/users/:** Ruta
- **123:** Path Params
- **?active=true&sort=true:** Query Params
- **#section1:** Fragment

Path Params: Parámetros de Ruta. Identifican recursos específicos

Query Params: Parámetros de Consulta

- Va después de ?
- Se usan para filtrar, ordenar, paginar

5 – CORS

CORS: Cross Origin Resource Sharing

- Mecanismo de Seguridad que restringe las peticiones HTTP entre diferentes orígenes
- Sirve para bloquear peticiones desde otro dominio que no sea el de la página actual

Headers:

- **Access-Allow-Control-Origin:** Dominio de la petición permitido
- **Access-Allow-Control-Methods:** Métodos de petición permitidos
- **Access-Allow-Control-Headers:** Cabeceras permitidas en la petición

Tema 4 – Framework Express

1 – Introducción Express

Express: Framework que simplifica la creación de servidores HTTP

Características:

- **Routing Declarativo:** Define rutas de forma clara y organizada
- **Middleware:** Sistema Flexible para procesar solicitudes
- **Alto Rendimiento:** Capa mínima de Node.js
- **Extensible:** Gran ecosistema de middleware
- **Version:** 5.0

2 – Primera Aplicación Express 5

Crear Servidor Express: `const app = express()`

Parsear JSON: `app.use(express.json())`

Lanzar Servidor (Escuchar Puerto): `app.listen(puerto, () => {})`

Ejecutar Servidor en Desarrollo: `npm run dev`

Ejecutar Servidor en Producción: `npm start`

3 – Routing

Métodos HTTP:

- **GET:** `app.get('/recurso', handler)`
- **POST:** `app.post('/recurso', handler)`
- **PUT:** `app.put('/recurso', handler)`
- **PATCH:** `app.patch('/recurso', handler)`
- **DELETE:** `app.delete('/recurso', handler)`
- **ALL:** `app.all('/recurso', handler)`

Parámetros de Ruta:

- **Leer parámetros de ruta:** `req.params`
- **Parámetro Obligatorio:**

```
app.get('/cursos/:id', (req, res) => {  
  const { id } = req.params;  
  res.json({ id });  
});
```

- **Múltiples Parámetros:**

```
app.get('/cursos/:categoria/:id', (req, res) => {  
  const { categoria, id } = req.params;  
  res.json({ categoria, id });  
});
```

- **Wildcard:**

```
app.get('/files/*filepath', (req, res) => {
  console.log(req.params.filepath); // Captura
});
```

Query Params: req.query

Respuestas:

- **Respuestas Tipo JSON:** res.json()
- **Status del JSON:** res.status()
- **Respuesta Sin Contenido:** res.end()
- **Respuesta:** res.end()
- **Respuesta con Archivo:** res.sendFile()
- **Redirección de rutas:** res.redirect()

4 – Routers Modulares

Crear Router: const router = Router()

Para usar un namespace: router.use('/client', router)

Rutas: router.get('ruta',middlewares, controllador)

Integracion En App:

```
// src/app.js
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import routes from './routes/index.js';
import { errorHandler, notFoundHandler } from './middleware/errorHandler.js';

const app = express();

// Seguridad
app.use(helmet());
app.use(cors());

// Parseo de body
app.use(express.json({ limit: '10kb' }));
app.use(express.urlencoded({ extended: true }));

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});

// Rutas de la API
app.use('/api', routes);

// Manejo de errores
app.use(notFoundHandler);
app.use(errorHandler);

export default app;
```


5 – Middleware

Middleware: Funciones que se ejecutan entre solicitud y respuesta

Middleware de Aplicación: `app.use(middleware)`

Middleware de Ruta: `app.use('/ruta', authMiddleware, rutas)`

Middleware Inline: `app.get('/admin', authMiddleware, adminMiddleware, handler)`

Middleware de Terceros: `app.use(helmet())`

Middleware de Error:

```
// Middleware de aplicación
app.use(express.json());
app.use(helmet());

// Middleware de ruta
app.use('/api', authMiddleware, routes);

// Middleware inline
app.get('/admin', authMiddleware, adminMiddleware, handler);

// Middleware de error (4 parámetros)
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: 'Error interno' });
});
```

Middleware Comunes:

```
export const logger = (req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
};
```

6 – Validación con Zod

Zod: Librería de validación

Esquemas de Validación:

```
export const updateCursoSchema = z.object({
  body: z.object({
    titulo: z.string().min(3).max(100).optional(),
    lenguaje: z.enum(['javascript', 'python', 'java', 'csharp']).optional(),
    nivel: z.enum(['basico', 'intermedio', 'avanzado']).optional(),
    descripcion: z.string().optional()
  }),
  params: z.object({
    id: z.string().regex(/^\d+$/, 'ID debe ser numérico')
  })
});
```

7 – Servir Archivos Estáticos y Logging

Middleware Para Archivos Estáticos: `express.static('carpeta')`

Morgan: Middleware de logging HTTP. Meter en `app.use()`

- **Logging Desarrollo:** `morgan('dev')`
- **Logging Produccion:** `morgan('combined')`
- **Logging Minimo:** `morgan('tiny')`

Tema 5 – MVC y BBDD NoSQL con MongoDB

1 – Patron MVC para APIs

MVC: Modelo Vista-Controlador

Componentes:

- **Modelo:** Estructura de Datos, validación, interacción con BD
- **Vista:** Respuesta al cliente
- **Controlador:** Lógica de Negocio, coordinación
- **Rutas:** Mapeo URL → Controlador

Ventajas:

- **Separación de Responsabilidades:** Cada componente tiene una función clara
- **Mantenibilidad:** Fácil de modificar partes específicas
- **Testabilidad:** Componentes probables de forma aislada
- **Escalabilidad:** Facilita el crecimiento del proyecto
- **Reutilización:** Código reutilizable entre endpoints

2 – MongoDB y Mongoose

MongoDB: Base de datos NoSQL orientada a documentos. Almacena JSONs

Mongoose: ODM que proporciona una capa de abstracción sobre MongoDB

Conexión a MongoDB: `mongoose.connect()`

Esquemas de Mongoose: `mongoose.Schema()`. Dentro se colocan los campos del documento

Modelos de Mongoose: `mongoose.model('nombreColeccion', esquema)`

Parametros de Cada Campo:

- **type:** Tipo de Dato del campo
- **required:** Campo requerido en un documento
- **unique:** Campo Único
- **lowercase:** Campo tiene que estar en minúsculas
- **trim:** Elimina espacios delante
- **match:** Patron que debe seguir
- **default:** Valor Por Defecto
- **min:** Valor mínimo
- **enum:** Enumerado de posibles valores
- **timestamps:** campos `createdAt` y `updatedAt`
- **versionKey:** Campo `__v`

3 – CRUD con MongoDB

CREATE:

- **Creado de Documentos:** `await User.create(data)`
- **Guardado de Documentos:** `User(data).save()`

RETRIEVE:

- **Busqueda de Todos:** `await User.find()`
- **Busqueda con Filtro:** `await User.find(filtro)`
- **Busqueda por ID:** `await User.findById(id)`
- **Busqueda un Documento:** `await User.findOne()`

UPDATE:

- **Actualizar Uno:** `await User.findByIdAndUpdate(id, cambios)`
- **Actualizar Varios:** `await User.updateMany(cambios)`

DELETE:

- **Eliminar Uno:** `await User.findByIdAndDelete(id)`
- **Eliminar Varios:** `await User.deleteMany(filtro)`

`lean()`: Optimiza consultadas de búsqueda mejorando el rendimiento

4 – Relaciones Entre Documentos

Documentos Embebidos: Incluir documentos dentro de otros

- **Ventajas:** Una sola consulta obtiene todo
- **Desventajas:** Datos Duplicados

Referencias: Almacenar IDs y usar `populate` para obtener datos

- **`.populate(campo referencia, campos a mostrar)`:** Muestra datos del documento referenciado

5 – Subida de Archivos con Multer

Multer: Middleware estándar para manejar multipart/form-data

multer.diskStorage(): Configuración de almacenamiento

- **destination:** directorio destino
- **filename:** nombre del fichero

Middleware de Ficheros:

- **fileFilter:** Listado de tipos de ficheros permitidos

```
const uploadMiddleware = multer({
  storage,
  fileFilter,
  limits: {
    fileSize: 10 * 1024 * 1024, // 10MB
    files: 5                     // Máximo 5 archivos
  }
});
```

uploadMiddleware.single(campo): Pasado de un fichero a un controlador

Tema 6 – Validación Avanzada y Manejos de Errores

1 – Validación Avanzada con Zod

Validación Condicional:

```
const createUserSchema = z.object({
  body: z.object({
    name: z.string().min(2).max(100),
    email: z.string().email(),
    password: z.string().min(8),
    userType: z.enum(['personal', 'business']),
    // Solo requerido si userType es 'business'
    companyName: z.string().optional(),
    taxId: z.string().optional()
  }).refine(
    (data) => {
      if (data.userType === 'business') {
        return data.companyName && data.companyName.length >= 2;
      }
      return true;
    },
    {
      message: 'Las cuentas business requieren nombre de empresa',
      path: ['companyName']
    }
  ).refine(
    (data) => {
      if (data.userType === 'business') {
        return data.taxId && data.taxId.length >= 8;
      }
      return true;
    },
    {
      message: 'Las cuentas business requieren CIF/NIF',
      path: ['taxId']
    }
  )
});
```

.transform(): Para hacer transformaciones de datos durante la validación

Esquemas Reutilizables:

```
export const objectIdSchema = z
  .string()
  .regex(/^[0-9a-fA-F]{24}$/, 'ID no válido');
```

2 – Soft Delete (Borrado Lógico)

Soft Delete: Marca registros como eliminados sin eliminarlos completamente

```
// Método de instancia: soft delete
schema.methods.softDelete = async function(deletedBy = null) {
  this.deleted = true;
  this.deletedAt = new Date();
  this.deletedBy = deletedBy;
  return this.save();
};
```

Hemos creado un plugin que añadiremos al esquema

schema.plugin(softDeletePlugin): Metemos el plugin en el esquema

3 – Sanitización de Datos

Tenemos que verificar las queries de mongo para prevenir inyecciones sql.

Ejemplos de Sanitización de Datos:

- Eliminar operadores peligrosos
- Limitar tamaño de los strings para prevenir DoS
- Eliminar campos no permitidos

Se aplica como un middleware

4 – Rate Limiting

Sirve para limitar el numero de peticiones

Se puede usar express-rate-limit para limitar.

Limite se aplica en las rutas

5 – Seguridad Con Helmet

Helmet configura headers HTTP para proteger tu aplicación de vulnerabilidades comunes.

7.2 ¿Qué protege Helmet?

Header	Protección
Content-Security-Policy	Previene XSS e inyección de datos
X-Content-Type-Options	Evita MIME sniffing
X-Frame-Options	Previene clickjacking
Strict-Transport-Security	Fuerza HTTPS
X-XSS-Protection	Filtro XSS del navegador
X-DNS-Prefetch-Control	Controla prefetch DNS

Tema 7 – Autenticación y Autorización con JWT

1 – Introducción a JWT

JWT: Estándar abierto para transmitir información de forma segura entre 2 partes como un objeto JSON

Partes:

- **Header:** Algoritmo y tipo de token
- **Payload:** Datos
- **Firma:** Firma de verificación

jsonwebtoken: Crear y verificar web token

bcryptjs: Hash de contraseñas

2 – Hash de Contraseñas

Almacenar contraseñas en texto plano es poco seguro. Mejor crear un hash

Hashear Contraseña: `await bcryptjs.hash(clearPassword,10)`. 10 es el salt

Comparar Contraseñas: `await bcryptjs.compare(clearPassword, hashedPassword)`

Salt: Valor aleatorio que se añade a la contraseña antes de hacer el hash

3 – Manejo de JWT

.env: Se debe añadir una clave a tu .env de 32 caracteres

Generación de Token: `jwt.sign()`

- Requiere usuario para generar payload con id y rol
- Requiere JWT_SECRET en el .env
- Requiere tiempo de expiración

Verificación de Token: `jwt.verify(token,SECRET)`

4 – Middleware de Autenticación

Función: Verifica el token y añade el usuario a `req.user`

Header: Authorization: Bearer TOKEN

Se protegen las rutas añadiendo el token

5 – Autorización de Roles

Autenticación: Token → Verifica quien eres

Autorización: Verifica que puedes hacer

Middleware de Rol: Comprueba el rol dentro de una lista de roles permitidos

Orden de Middlewares: Se debe hacer autenticación antes de autorización ya que se necesita req.user para obtener su rol

6 – Refresh Token

Refresh Token: Permiten obtener nuevos access token sin pedir credenciales

Refresh Token vs Access Token:

Access Token	Refresh Token
Vida Corta	Vida Larga
Se envía en cada petición	Solo se usa para renovar
Si se roba, poco tiempo de exposición	Se almacena de forma segura

Como generar: Con `crypto.randomBytes(64).toString('hex')`

Seguridad de Refresh Tokens:

- Almacenar con HTTPCookie
- Rotacion de Tokens
- Detección de reutilización
- Limitar tokens activos

Tema 8 – Documentación, Testing y Monitorización

1 – Documentación con Swagger

Swagger: Conjunto de herramientas de código abierto para diseñar, construir y documentar API REST. Sigue OPENAPI 3.0

Configuración: Crear con swaggerJsDoc(JSON_DE_DATOS)

- **definition:** Definición de la API
- **servers:** Servidores
- **componentes:** Componentes del api. Aquí va el token
- **schemas:** Esquemas de Modelos de datos utilizados

Integración: Despues de los middlewares, antes de las rutas

Documentación de Rutas:

```
/**
 * @openapi
 * /api/client/{id}:
 * delete:
 *   tags:
 *     - Client
 *   summary: Eliminar cliente
 *   description: Elimina un cliente (soft o hard delete según query param)
 *   security:
 *     - bearerAuth: []
 *   parameters:
 *     - in: path
 *       name: id
 *       required: true
 *       schema:
 *         type: string
 *         description: ID del cliente a eliminar
 *     - in: query
 *       name: soft
 *       schema:
 *         type: boolean
 *         example: true
 *         description: Si es true realiza borrado lógico
 *   responses:
 *     200:
 *       description: Cliente eliminado
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             properties:
 *               message:
 *                 type: string
 *                 example: Usuario eliminado
 *     400:
 *       description: Datos inválidos o petición invalida
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/Error'
```

- **/api/client/{id}**: Ruta documentada
- **delete**: Método HTTP
- **tags**: Modelo de datos utilizados
- **security**: Seguridad de la ruta
- **parameters**: Parametros de ruta o query params
 - **in**: Si esta en query params o en ruta
 - **name**: Nombre de parámetro
- **requestBody**: Body necesario
- **responses**: Tipos de respuesta que puede dar la ruta
 - **200/400**: Código de respuesta

2 – Testing Con Jest

Jest: Framework de Testing

- Zero configuración
- Mocking Integrado
- Coverage de Código
- Watch Mode para el desarrollo

Es necesario exportar app

describe(nombre, () => {}): Describe un grupo de tests

beforeEach(() => {}): Funcionalidad a implementar antes de cada test

it(): Test individual

- **expect():** Resultado que espera
- **.toBe():** Valor que tiene que tener un campo
- **toHaveProperty():** Propiedad que debe tener un campo

npm test: Ejecuta test

npm test:coverage: Devuelve el porcentaje de código cubierto

3 – Monitorización Con Slack

Slack:

- Notificaciones a tiempo real
- Historial de errores centralizado
- Alertas al equipo de desarrollo

Configurar WebHook:

- **Linkado a Slack:** IncomingWebhook(SLACK_WEBHOOK)
- **Enviado a Slack:** webhook.send()

Integrar en app:

```
import morganBody from 'morgan-body';
import { loggerStream } from './utils/handleLogger.js';

// Después de express.json(), antes de las rutas
morganBody(app, {
  noColors: true,
  skip: (req, res) => res.statusCode < 400, // Solo errores
  stream: loggerStream
});
```

Tema 9 – Supabase y PostgreSQL

1 – Introducción a Supabase

Supabase: Alternativa open-source a Firebase

- Base de Datos PostgreSQL
- Autenticación integrada
- Storage para archivos
- Realtime suscripción
- Edge Functions (serverless)
- API REST automática generada desde tu esquema

2 – PostgreSQL vs MongoDB

Terminología:

Descripción	MongoDB	PostgreSQL
Contenedor principal	Database	Database
Grupo de Datos Relacionados	Colecciones	Table
Un registro individual	Document	Row/Record
Un atributo de registro	Field	Column
Identificador Unico	id	id
Datos Anidados	Documentos Embebidos	JSON columna
Relación entre tablas	Referencias	Foreign Keys

Esquemas:

```
model User {  
  id    Int    @id @default(autoincrement())  
  name  String  
  email String  @unique  
  posts Post[] // Relación automática  
}
```

3 – Prisma: El ORM Moderno

Características:

- **Type-safe:** Autocompletado y validación en tiempo de compilación
- **Migraciones Automática:** Gestión de esquemas integrada
- **Prisma:** GUI para explorar datos
- **Rendimiento:** Queries optimizadas
- **Multi-database:** PostgreSQL, MySQL, SQLite, MongoDB, SQL Server

4 – Modelo de Datos

Tipos de Datos:

- **Identificadores:**
 - **Autoincremento:** id Int @id @default(autoincrement())
 - **UUID:** id String @id @default(uuid())
 - **CUID:** id String @id @default(cuid())
- **Cadenas:**
 - **Cadenas:** name String
 - **Nullable:** bio String?
 - **Text:** content String @db.Text
- **Numeros:**
 - **Enteros:** age Int
 - **Coma Flotante:** price Float
 - **Precisión Exacta:** amount Decimal @db.Decimal(10,2)
- **Booleanos y Fechas:**
 - **Boolean:** active Boolean @default(true)
 - **Fechas:** createdAt DateTime @default(now())
- **JSON:** metadata Json?
- **Enums:** role NOMBRE_ENUM

Atributos Importantes:

- **Clave Primaria:** @id
- **Valor Unico:** @unique
- **Valor Por Defecto:** @default()
- **Auto-actualizar timestamp:** @updatedAt
- **Relaciones Foreign Key:** @relation

5 – CRUD

CREATE:

- **Crear:** await prisma.tabla.create(datos)
- **Crear Relacion:** await prisma.tabla.create({campo : {connect : {id: id_new}}})
- **Multiples Registro:** await prisma.tabla.createMany(data)

RETRIEVE:

- **Busqueda de un documento:** await prisma.tabla.findUnique(filtro)
- **Busqueda Uno si existe o error:** await prisma.tabla.findUniqueOrThrow(filtro)
- **Busqueda Primero que Encuentra:** await prisma.tabla.findFirst(filtro)
- **Buscar Varios:** await prisma.tabla.findMany()

UPDATE:

- **Actualizar Uno:** `await prisma.tabla.update({where: filtro, data: cambios})`
- **Actualizar o Crear:** `await prisma.tabla.upsert({where: filtro, update: cambios, create: datos})`
- **Actualizar Muchos:** `await prisma.tabla.updateMany({where: filtro, data: cambios})`

DELETE:

- **Borrar Uno:** `await prisma.tabla.delete(filtro)`
- **Borrar Muchos:** `await prisma.tabla.deleteMany(filtro)`

6 – Relaciones en PostgreSQL

One-To-Many (1:N): Uno documento relacionado varios

Many-To-Many (N:M): Varios documentos relacionados entre si de ambas tablas

One-to-One (1:1): Un documento relacionado con solo otro documento

7 – Migraciones y Prisma Studio

Comando: `npx prisma migrate dev --name init`

- Genera SQL de migración
- Aplica cambios a la BD
- Regenera el cliente Prisma

Tema 10 – WebSockets – Comunicación IRT

1 - ¿Qué son los WebSockets?

WebSocket: Protocolo de Comunicación que proporciona canales de comunicación full-duplex sobre una única conexión TCP

En HTTP la conexión se cierra después de cada response/request

En WebSocket la conexión permanece abierta y ambos puede enviar datos

Socket: Herramienta del sistema operativo para intercomunicar procesos

- **Conexión Persistente:** Se abre y permanece abierta
- **Full Duplex:** Comunicación bidireccional simultanea
- **Bajo Overhead:** Sin headers http en cada mensaje
- **Tiempo Real:** Latencia Minima

2 – HTTP vs WebSocket

Característica	HTTP	WebSocket
Conexión	Nueva por cada request	Persistente
Dirección	Unidireccional (cliente inicia)	Bidireccional
Overhead	Headers en cada request (~800 bytes)	Solo datos (~2 bytes)
Estado	Stateless	Stateful
Protocolo	http:// / https://	ws:// / wss://
Puerto	80 / 443	80 / 443 (mismo puerto)
Estado	Stateless	Stateful

3 – Socket.IO

Configuración Servidor Socket:

- **Creación de Servidor Socket:** `const io = new Server(servidorHttp, {detalles})`
- **Evento de Conexión:** `io.on('connection', (socket) => {})`
- **Evento Personalizado:** `socket.on('evento', () => {})`

Métodos de Emisión:

- **A un socket específico:** `socket.emit('evento', datos)`
- **A todos menos emisor:** `socket.broadcast.emit('evento', datos)`
- **A todos los conectados:** `io.emit('evento', datos)`
- **A un socket por id:** `io.to(socketId).emit('evento', datos)`
- **A una sala específica:** `io.to('sala').emit('evento', datos)`

4 – Rooms y Namespaces

Salas: Permite agrupar sockets para enviar mensajes a un subconjunto de clientes

- **Unirse a Sala:** `socket.join('sala')`

Namespaces: Dividen la lógica en diferentes endpoints

- **Crear Namespace:** `io.of()`

Tema 11 – Deploy y DevOps

1 – Preparar para Producción

Aspecto	Development	Production
NODE_ENV	development	production
Logs	Verbose, queries	Solo errores
Errores	Stack traces	Mensajes genéricos
Hot reload	Sí	No
Minificación	No	Sí
HTTPS	Opcional	Obligatorio
Base de datos	Local/dev	Cloud/managed

2 – Docker

Características:

- **Consistencia:** Mínimo entorno en desarrollo, testing y producción
- **Aislamiento:** Dependencias no conflictan
- **Portabilidad:** Funciona en cualquier maquina con docker
- **Escalabilidad:** Facil de replicar contenedores

Dockerfile Multi-Stage:

- **Fase 1: Dependencias**

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
```

- **FROM node:22-alpine:** Entorno de node
- **WORKDIR /app:** Directorio de trabajo
- **COPY:** Copiado de package.json y package-lock.json
- **RUN:** Instalacion de dependencias en producción

- **Fase 2: Producción**

```
FROM node:22-alpine AS runner
WORKDIR /app

RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nodeapp

COPY --from=deps /app/node_modules ./node_modules
COPY --chown=nodeapp:nodejs . .

USER nodeapp

ENV NODE_ENV=production
EXPOSE 3000

CMD ["npm", "start"]
```

- **RUN:** Ejecutar comando
- **ENV:** Añadir a entorno que estamos en producción
- **EXPOSE:** Expone puerto
- **CMD [“npm”, “start”]:** Lanza en producción

Comandos:

- **Construir Imagen:** `docker build -t mi-api`
- **Listar imágenes:** `docker images`
- **Ejecutar Contenedor:** `docker run -d -p 3000:3000 --name mi-api mi-api`

3 – Docker Compose

Docker-Compose: Herramienta para definir y ejecutar aplicaciones multicontrolador

```
services: You, 5 days ago • Docker Configurado ...
  > Run Service
  api:
    build: .
    container_name: bildy-app
    ports:
      - "3000:3000"
    env_file:
      - .env
    depends_on:
      - mongo
    volumes:
      - ./app
      - /app/node_modules

  > Run Service
  mongo:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

volumes:
  mongo_data:
```

Comando:

- **Iniciar Servicios:** `docker compose up`
- **Reconstruir Imágenes:** `docker compose up --build`
- **Parar Servicios:** `docker compose down`

4 – CI/CD con GitHub Actions

CI: Ejecutar los tests automáticamente en cada push

CD: Desplegar automáticamente cuando pasan los test

```
name: Tests

You, 5 days ago | 1 author (You)
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

You, yesterday | 1 author (You)
jobs:
  test:
    runs-on: ubuntu-latest

    timeout-minutes: 15

    steps:
      - name: Checkout código
        uses: actions/checkout@v4
        with:
          fetch-depth: 1

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependencias (CI)
        run: npm ci

      - name: Ejecutar tests
        run: npm run test

      env:
        NODE_ENV: test
        JWT_SECRET: test-secret-32-characters-long!!
```

Tema 12 – TypeScript con Node.js

1 – TypeScript

Ventajas:

Característica	JavaScript	TypeScript
Tipado	Dinámico	Estático
Errores	En runtime	En compilación
Autocompletado	Limitado	Excelente
Refactoring	Riesgoso	Seguro
Documentación	Manual	Integrada en tipos
Curva aprendizaje	Baja	Media

2 – Tipos Básico

Tipos Primitivos:

- **Cadenas:** const name: String
- **Numbers:** const age: Number
- **Boolean:** const isActive: Boolean
- **Null:** const nothing : null
- **Undefined:** const notDef : undefined
- **Flexible:** let flex: any
- **Unknown (mas seguro):** let userInput: unknown

Arrays y Tuplas:

- **Arrays:**
 - const numbers: number[]
 - const numbers: Array<number>
- **Arrays Readonly:** const readonlyArray : readonly number[]
- **Tuplas:**
 - const coordinate: [number, number]
 - const userRecord: [number, string, boolean]
- **Tuplas con labels:**
 - type Point = [x: number, y: number];
 - const point: Point = [10, 20];

Objetos: const user: { name: string; age: number } = {data}